

CarRent + Spring Security + JW

JWT, logowanie i zabezpieczenie endpointów

Authentication

Authorization

Servlet filters

- src
 - main
 - java
 - com.umcsuser.carrent
 - db
 - dto
 - LoginRequest
 - LoginResponse
 - RentalRequest
 - models
 - repositories
 - security
 - JwtAuthFilter
 - JwtUtil
 - MyUserDetailsService
 - SecurityConfig
 - services
 - web
 - CarRentApplication
 - resources
 - application.yml

Zmiana w projekcie

Dodanie logowania i kontrola dostępu.

Bez security

Endpointy REST są dostępne bez tokenu. Klient może podać userId w URL i wykonać operację jako ktoś inny.

Security

Klient loguje się, dostaje token JWT, a backend ustala użytkownika na podstawie tokenu, nie na podstawie body albo URL.



Trzy główne pojęcia

Spring Security łączy uwierzytelnianie, autoryzację i filtry servletowe.

Authentication

Login + hasło.
Po poprawnym logowaniu backend generuje JWT.

Authorization

np. USER może wypożyczyć i oddać pojazd.
ADMIN może dostać dodatkowe endpointy.

Servlet filters

Co dzieje się przed kontrolerem?

Filtr sprawdza nagłówek Authorization i ustawia SecurityContext.

Filtr wykonuje się zanim request trafi do metody kontrolera.

JWT

Najpierw logowanie, potem każdy chroniony request przynosi token.

1. Logowanie

POST /api/auth/login
body: { login, password }

2. Odpowiedź

Backend zwraca token JWT
{ "token": "eyJ..." }

3. Request chroniony

POST /api/rentals/rent
Authorization: Bearer <token>

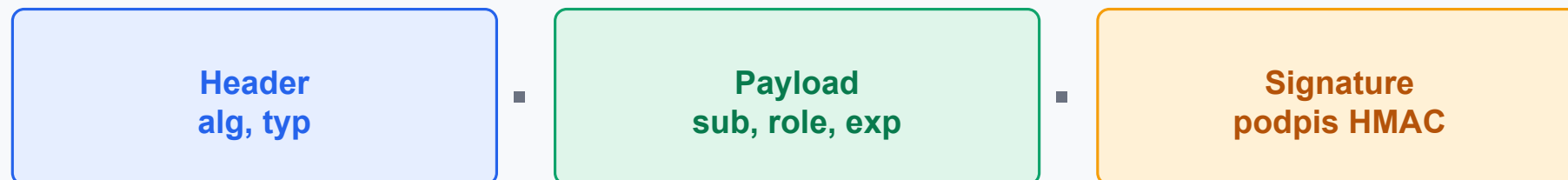
4. Controller

JwtAuthFilter ustawia użytkownika
Controller używa @AuthenticationPrincipal

JWT nie zastępuje hasła. Hasło jest potrzebne tylko przy logowaniu.

JWT: co jest w tokenie?

Token ma trzy części: header, payload i signature.



```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoia2FtaWwiLCJpYXQiOiE3Nzk2NjUyMjIsImV4cCI6MTc3OTY2ODgyMn0.5HzV3rJNoaU50tTbc_s-ZdiRaMrZSsmCcQvblp1Qvbk
```

**Podpis sprawdza, czy token nie został zmieniony.
Payload można odczytać - nie wkładamy tam haseł ani sekretów.**

Krok 1: zależności w POM

Dodajemy Spring Security i bibliotekę JJWT do generowania oraz parsowania tokenów.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

Samo dodanie startera zmienia zachowanie aplikacji: Spring zaczyna chronić endpointy.

Dlatego od razu dodajemy SecurityConfig, żeby jawnie powiedzieć, co jest publiczne, a co wymaga tokenu.

Krok 2: application.yml

Konfiguracja tokenu powinna pochodzić z ustawień aplikacji, nie z kodu Javy.

```
spring:
  profiles:
    active: ${APP_PROFILE:json}
  main:
    banner-mode: off

server:
  port: ${PORT:8080}

jwt:
  secret: ${JWT_SECRET:5Sy1-R-Zj7ajgcabswkEttARiS8Q4Vs9IhYCVAt1bQ0}
  expiration: ${JWT_EXPIRATION:3600000}
```

JWT_SECRET produkcyjnie - zmienna środowiskowa.

expiration = 3600000 ms, czyli 1 godzina ważności tokenu.

```
{
  "typ": "JWT",
  "alg": "HS256"
}
```

Decoded Payload

JSON Claims Breakdown

```
{
  "role": "ROLE_USER",
  "sub": "kamil",
  "iat": 1779697505,
  "exp": 1779701105
}
```

JWT Signature Verification (Optional)

Enter the secret used to sign the JWT below:

BASE64URL ENCODED ☒

> Secret

5Sy1-R-Zj7ajgcabswkEttARiS8Q4Vs9IhYCVAt1bQ0

✓ Valid secret

Nowe komponenty

Dodajemy warstwę security obok istniejących kontrolerów i serwisów.

SecurityConfig

reguły dostępu + filter chain

JwtUtil

generowanie i walidacja tokenu

JwtAuthFilter

odczyt nagłówka Authorization

MyUserDetailsService

adapter User -> UserDetails

AuthController

endpoint /api/auth/login

DTO

LoginRequest, LoginResponse, RentalRequest

RentalController

rent/return jako zalogowany użytkownik

SecurityContext

informacja kto wykonuje request

SecurityConfig 1/2

Konfigurujemy łańcuch filtrów i reguły dostępu do endpointów.

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain securityFilterChain(
        HttpSecurity http,
        JwtAuthFilter jwtAuthFilter,
        AuthenticationProvider authProvider
    ) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .formLogin(form -> form.disable())
            .httpBasic(basic -> basic.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
    }
}
```

/api/auth/**
logowanie jest publiczne

/api/admin/**
wymaga roli ADMIN

anyRequest()
każdy inny endpoint wymaga tokenu

SecurityConfig 2/2

JWT oznacza aplikację bez sesji serwerowej.

```
.sessionManagement(session ->
    session.sessionCreationPolicy(
        SessionCreationPolicy.STATELESS
    )
)
.authenticationProvider(authProvider)
.addFilterBefore(
    jwtAuthFilter,
    UsernamePasswordAuthenticationFilter.class
);

return http.build();
```

```
@Bean
public AuthenticationProvider authenticationProvider(
    UserDetailsService userDetailsService,
    PasswordEncoder passwordEncoder
) {
    DaoAuthenticationProvider provider =
        new DaoAuthenticationProvider();
    provider.setUserDetailsService(userDetailsService);
    provider.setPasswordEncoder(passwordEncoder);
    return provider;
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

STATELESS

Serwer nie pamięta sesji. Każdy request musi mieć token.

DaoAuthenticationProvider

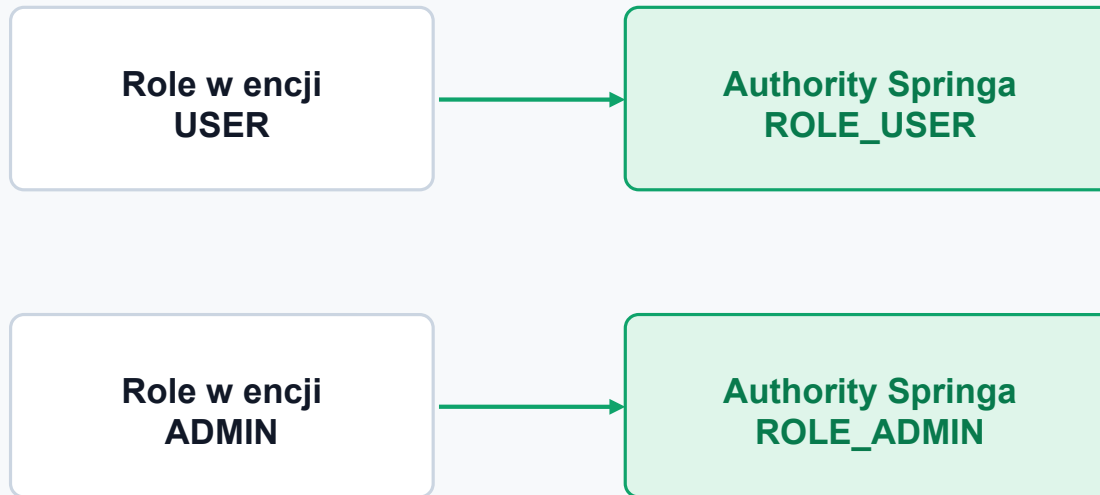
Używa UserDetailsService i PasswordEncoder do logowania.

addFilterBefore

Nasz filtr JWT działa przed standardowym filtrem logowania.

Role w naszej aplikacji

W modelu domenowym rola to USER albo ADMIN, a Spring sprawdza authority.



```
new SimpleGrantedAuthority("ROLE_" +  
user.getRole().name())  
  
.requestMatchers("/api/admin/  
**").hasRole("ADMIN")
```

MyUserDetailsService

Adapter między naszym UserRepository a Spring Security.

```
@Service
@RequiredArgsConstructor
public class MyUserDetailsService implements UserDetailsService {
    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {
        User user = userRepository.findByLogin(username)
            .orElseThrow(() -> new UsernameNotFoundException(username));

        List<GrantedAuthority> authorities = List.of(
            new SimpleGrantedAuthority("ROLE_" + user.getRole().name())
        );

        return new org.springframework.security.core.userdetails.User(
            user.getLogin(),
            user.getPasswordHash(),
            authorities
        );
    }
}
```

**Spring nie używa bezpośrednio naszej
encji User.
Potrzebuje obiektu UserDetails.**

adapter: nasz model -> model security.

DTO do logowania

Do API nie wysyłamy encji User, tylko małe obiekty wejścia i wyjścia.

```
public record LoginRequest(  
    String login,  
    String password  
) { }  
  
public record LoginResponse(  
    String token  
) { }
```

Request

POST /api/auth/login
{ "login": "lukasz", "password": "lukasz123" }

Response

{ "token": "..." }

**Hasło pojawia się tylko w requestcie logowania.
W kolejnych requestach używamy tokenu.**

AuthController - logowanie

AuthenticationManager sprawdza login i hasło, a JwtUtil generuje token.

```
@RestController
@RequestMapping("api/auth")
@RequiredArgsConstructor
public class AuthController {
    private final AuthenticationManager authenticationManager;
    private final JwtUtil jwtUtil;

    @PostMapping("/login")
    public ResponseEntity<LoginResponse> login(
        @RequestBody LoginRequest loginRequest) {
        Authentication auth;
        try {
            auth = authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(
                    loginRequest.login(),
                    loginRequest.password()
                ));
        } catch (BadCredentialsException e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
        }

        UserDetails userDetails = (UserDetails) auth.getPrincipal();
        String token = jwtUtil.generateToken(userDetails);
        return ResponseEntity.ok(new LoginResponse(token));
    }
}
```

Jeżeli login lub hasło są błędne, authenticate rzuca BadCredentialsException.

Jeżeli dane są poprawne, principal to UserDetails z loginem i rolami.

JwtUtil 1/2 - generowanie tokenu

Token dostaje login jako subject, rolę jako claim i datę wygaśnięcia.

```
@Component
public class JwtUtil {
    @Value("${jwt.secret}")
    private String secretKey;

    @Value("${jwt.expiration}")
    private long expirationMs;

    public String generateToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("role", getUserRole(userDetails));

        Date now = new Date();
        Date expirationDate = new Date(now.getTime() + expirationMs);

        return Jwts.builder()
            .header().type("JWT").and()
            .claims().add(claims).and()
            .subject(userDetails.getUsername())
            .issuedAt(now)
            .expiration(expirationDate)
            .signWith(getSigningKey())
            .compact();
    }
}
```

subject = login użytkownika

**claim role = ROLE_USER albo
ROLE_ADMIN**

**expiration = kiedy token przestaje
działać**

JwtUtil 2/2 - odczyt i walidacja

Sprawdzamy podpis, login użytkownika i czas wygaśnięcia.

```
public boolean validateToken(String token, UserDetails userDetails) {
    try {
        final String username = extractUsername(token);
        if (!username.equals(userDetails.getUsername())) {
            return false;
        }
        return !getClaims(token).getExpiration().before(new Date());
    } catch (Exception e) {
        return false;
    }
}

public String extractUsername(String token) {
    return getClaims(token).getSubject();
}

private Claims getClaims(String token) {
    Jws<Claims> jwtClaims = Jwts.parser()
        .verifyWith(getSigningKey())
        .build()
        .parseSignedClaims(token);
    return jwtClaims.getPayload();
}

private SecretKey getSigningKey() {
    byte[] keyBytes = Decoders.BASE64URL.decode(secretKey);
    return Keys.hmacShaKeyFor(keyBytes);
}
```

verifyWith(...) sprawdza podpis tokenu.

**HMAC nie ukrywa payloadu.
HMAC potwierdza integralność i autentyczność podpisu.**

Jeżeli token jest stary, zmieniony albo podpisany innym kluczem - walidacja zwraca false.

JwtAuthFilter 1/3 - wejście do filtra

Filtr uruchamia się przed kontrolerem i szuka nagłówka Authorization.

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    @Autowired
    private JwtUtil jwtUtil;

    @Autowired
    private UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain
    ) throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            // brak nagłówka lub niepoprawny format
            filterChain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);
```

@Component
Spring tworzy filtr jako bean.

@Autowired
Spring wstrzykuje JwtUtil i
UserDetailsService.

**Brak Bearer = request idzie dalej,
ale bez zalogowanego
użytkownika.**

JwtAuthFilter 2/3 - walidacja tokenu

Jeśli token jest poprawny, tworzymy obiekt Authentication.

```
try {
    String username = jwtUtile.extractUsername(token);

    if (username != null &&
        SecurityContextHolder.getContext().getAuthentication() == null) {

        UserDetails userDetails =
            userDetailsService.loadUserByUsername(username);

        if (jwtUtile.validateToken(token, userDetails)) {
            UsernamePasswordAuthenticationToken authToken =
                new UsernamePasswordAuthenticationToken(
                    userDetails,
                    null,
                    userDetails.getAuthorities()
                );

            authToken.setDetails(
                new WebAuthenticationDetailsSource().buildDetails(request)
            );

            SecurityContextHolder.getContext().setAuthentication(authToken);
        }
    }
} catch (Exception e) {
    // brak Authentication w SecurityContext
}

filterChain.doFilter(request, response);
```

extractUsername()
odczytuje subject z JWT.

loadUserByUsername()
pobiera aktualne dane usera z repo.

validateToken()
sprawdza podpis, login i expiration.

setAuthentication()
to moment zalogowania requestu.

RentalController

Najważniejsza zmiana: `userId` nie pochodzi od klienta.

```
// Przed security:
@PostMapping("/users/{userId}/rent/{vehicleId}")
public Rental rent(@PathVariable String userId,
                  @PathVariable String vehicleId) {
    return rentalService.rentVehicle(userId,
    vehicleId);
}
```

```
// Po security:
@PostMapping("/rent")
public ResponseEntity<Rental> rent(
    @RequestBody RentalRequest rentalRequest,
    @AuthenticationPrincipal UserDetails userDetails
) {
    String login = userDetails.getUsername();
    User user = userService.findByLogin(login);
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(rentalService.rentVehicle(
            user.getId(), rentalRequest.vehicleId()));
}
```

@AuthenticationPrincipal = użytkownik ustawiony przez **JwtAuthFilter** w **SecurityContext**.

Endpoint: wypożyczenie pojazdu

Body zawiera tylko vehicleId. User pochodzi z tokenu.

```
@PostMapping("/rent")
public ResponseEntity<Rental> rent(
    @RequestBody RentalRequest rentalRequest,
    @AuthenticationPrincipal UserDetails userDetails
) {
    String login = userDetails.getUsername();
    User user = userService.findByLogin(login);

    Rental rental = rentalService.rentVehicle(
        user.getId(),
        rentalRequest.vehicleId()
    );

    return ResponseEntity.status(HttpStatus.CREATED).body(rental);
}
```

Request

POST /api/rentals/rent
Authorization: Bearer <token>

```
{
  "vehicleId": "3"
}
```

Endpoint: zwrot pojazdu

Nie wysyłamy body, bo aktywne wypożyczenie szukamy po zalogowanym userze.

Do zaimplementowania :)

Request

POST /api/rentals/return
Authorization: Bearer <token>

Body

none - nic nie wysyłamy jeżeli tylko jeden pojazd na raz można wypożyczyć

**Jeżeli user nie ma aktywnego wypożyczenia,
serwis rzuci wyjątek biznesowy.**

Logika zwrotu zostaje w serwisie

Controller obsługuje HTTP, a RentalService zna regułę biznesową.

```
@Override
public Rental returnVehicle(String userId) {
    //...
}
```

1. Znajdź rental

Po userId i aktywnym wypożyczeniu.

2. Ustaw datę zwrotu

returnDateTime kończy wypożyczenie.

3. Zapisz

Repozytorium wybiera profil danych.

Test w Postmanie: logowanie

Najpierw bierzemy token z endpointu /api/auth/login.

POST

http://localhost:8080/api/auth/login

```
{  
  "login": "lukasz",  
  "password": "lukasz123"  
}
```

200 OK

Backend zwraca LoginResponse z tokenem.

```
{  
  "token": "..."  
}
```

W kolejnych requestach w Postmanie: Authorization -> Type: Bearer Token -> wklejamy sam token.

Test w Postmanie: rent i return

Oba endpointy wymagają tokenu w nagłówku Authorization.

Wypożyczenie

POST /api/rentals/rent

Authorization: Bearer <token>
Content-Type: application/json

```
{  
  "vehicleId": "3"  
}
```

Zwrot

POST /api/rentals/return

Authorization: Bearer <token>

Body: none

Rent: body z vehicleId.
Return: bez body, bo serwis szuka aktywnego wypożyczenia po userId z tokenu.

Statusy HTTP i typowe błędy

Te odpowiedzi pomagają szybko znaleźć problem w konfiguracji security.

401 Unauthorized

Brak tokenu, zły token albo token wygaś.
Request nie jest uwierzytelniony.

403 Forbidden

User jest znany, ale nie ma wymaganej roli

409 Conflict

Błąd biznesowy: np. brak aktywnego wypożyczenia albo pojazd jest już zajęty.

- brak nagłówka Authorization albo brak słowa Bearer
- endpoint odpalony jako GET zamiast POST

checklist implementacji

Kolejność kroków do przećwiczenia na repozytorium CarRent.

1	POM	dodać spring-boot-starter-security oraz jjwt-api/impl/jackson
2	Konfiguracja	dodać jwt.secret i jwt.expiration w application.yml
3	Security	dodać SecurityConfig, JwtUtil, JwtAuthFilter, MyUserDetailsService
4	Logowanie	dodać LoginRequest, LoginResponse i AuthController
5	Rent/return	użyć @AuthenticationPrincipal zamiast userId od klienta
6	Testy	sprawdzić login, rent, return i typowe 401/403

Podsumowanie

Spring Security w CarRent to przede wszystkim kontrola wejścia do aplikacji.

Najważniejsze

- JWT token dostajemy po logowaniu
- filtr działa przed kontrolerem
- SecurityContext przechowuje usera requestu

W kodzie

- SecurityConfig definiuje reguły
- UserDetailsService mapuje role
- JwtUtil generuje i sprawdza token

W API

- login zwraca token
- rent używa vehicleId z body
- return działa bez body

userId do operacji użytkownika bierzemy z tokenu, nie od klienta.



Dziękuję za uwagę

CarRent + Spring Security + JWT

